Trading Bot — Product Requirements Document

Project type: Personal / Fun
Owner: Sam
Status: Draft v1.0
Last updated: May 2026

Problem Statement

Building a trading bot by hand is the fastest way to deeply understand market microstructure, API design, async event loops, and systematic strategy development — things that are hard to learn from reading alone. This project exists purely as a technical learning exercise and sandbox, with no expectation of real-world profit. The "problem" it solves is the gap between knowing about algo trading conceptually and actually building and running one end-to-end.

Goals

1. **Build a working bot** — executes real or paper trades autonomously based on at least one defined strategy.
2. **Observable and debuggable** — every decision the bot makes is logged and explainable after the fact.
3. **Pluggable strategies** — adding a new strategy requires no changes to the core engine; just implement the interface.
4. **Survive a full market session** — runs continuously for a full trading day without crashing or entering an undefined state.
5. **Learns something real** — forces hands-on engagement with rate limits, order types, slippage, and data quality issues.

Non-Goals

Non-Goal	**Rationale**
Real money trading (v1)	Paper trading first — risk is real, fun project doesn't need financial exposure

Non-Goal	Rationale
ML/predictive modelling	Adds enormous complexity; rule-based strategies are sufficient for learning the architecture
Multi-asset class support (options, futures, crypto)	Scope control — equities or a single crypto pair is sufficient
Web UI / dashboard	Terminal output + logs are fine for v1; a UI is a separate project
Portfolio management / position sizing algorithms	Kelly criterion etc. is a rabbit hole; flat position sizing is fine initially
Tax reporting or trade journaling	Out of scope entirely

User Stories

Since this is a solo project, "user" = you (the developer/operator).

As the operator, I want to:

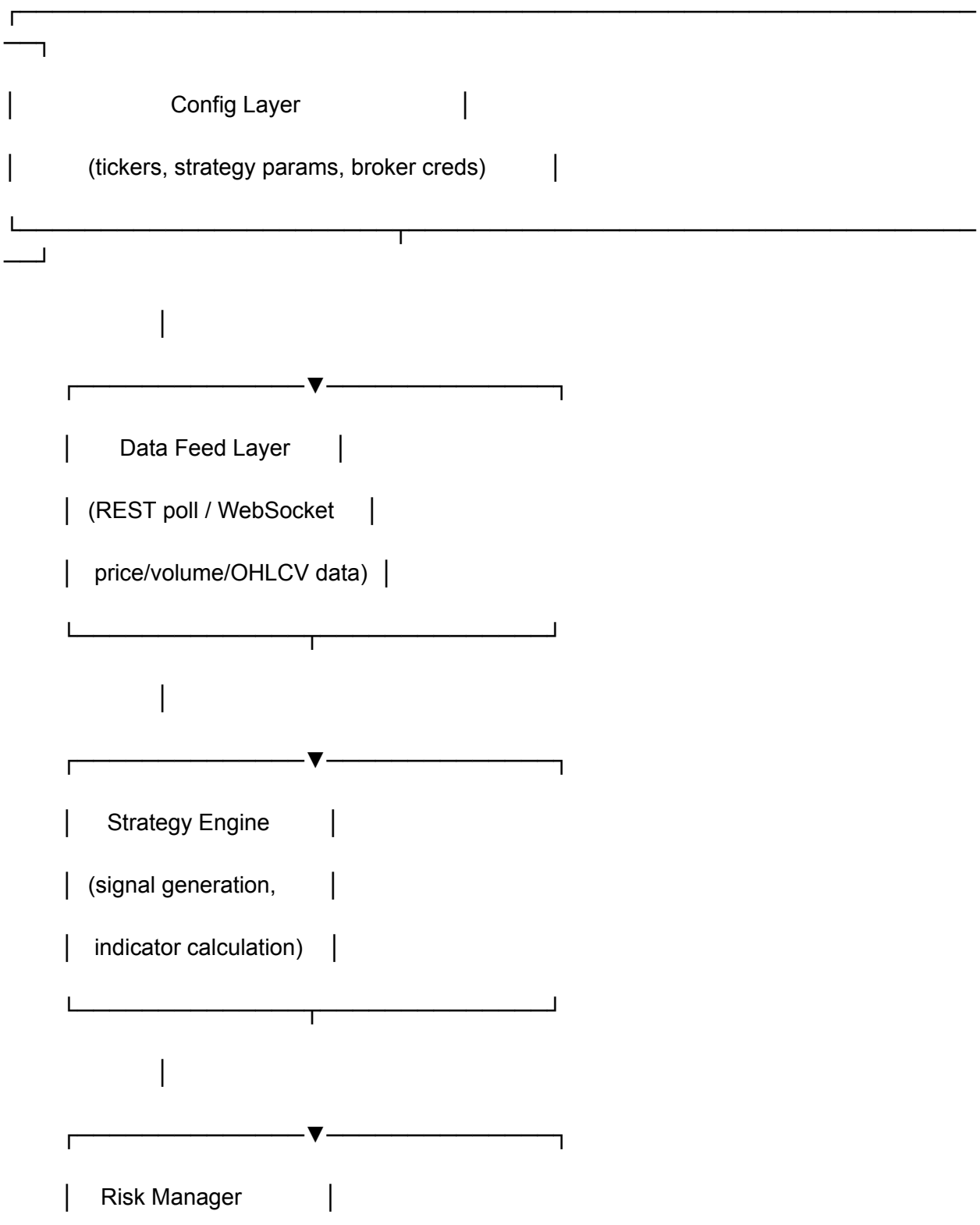
- Connect the bot to a paper trading account so I can test without financial risk
- Define a strategy in one file and have the bot pick it up automatically
- See a live feed of what the bot is doing (signals, decisions, order placement) in the terminal
- Review a log of all trades and the reasoning behind each one after the session ends
- Configure which tickers the bot watches from a single config file
- Halt the bot immediately (kill switch) without leaving open positions
- Replay historical data through a strategy to backtest it before running live

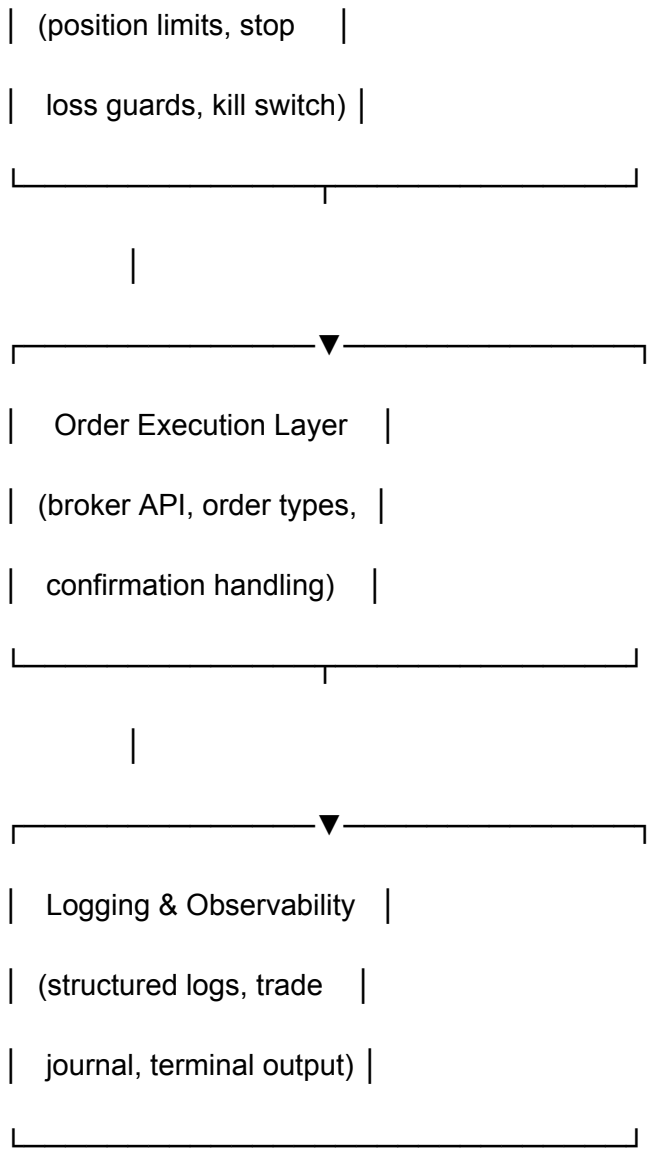
As a curious developer, I want to:

- Understand how the strategy engine is structured so I can swap strategies easily
- See clear separation between data ingestion, signal generation, and order execution
- Have tests I can run against the strategy logic without needing a live connection

---

System Architecture Overview





Requirements

P0 — Must Have (v1 ships with these)

1. Broker Integration (Paper Trading)

- Connect to a paper trading account via a well-documented API (Alpaca recommended — free paper trading, clean REST + WebSocket API, UK-accessible)
- Authenticate using API key/secret stored in environment variables, never hardcoded
- Place market orders and limit orders

- Retrieve current positions and open orders
- **Acceptance criteria:**
  - `GET /v2/account` returns account details without error
  - A test market order is placed, confirmed, and reflected in positions
  - Credentials are read from `.env`, never from source code

2. Data Feed

- Pull OHLCV bar data (at minimum 1-minute bars) for a configurable list of tickers
- Support both historical data (for backtesting) and live/streaming data (for live trading)
- Handle API rate limits gracefully with exponential backoff
- **Acceptance criteria:**
  - Can request 30 days of 1-minute bars for any ticker
  - Live price updates arrive within 2 seconds of market movement
  - Rate limit errors are caught, logged, and retried — never crash the bot

3. Strategy Interface

- Define a clear `Strategy` base class or protocol with:
  - `name: str`
  - `on_bar(bar: Bar) -> Signal | None`
  - `on_start() -> None`
  - `on_stop() -> None`
- **Acceptance criteria:**
  - A new strategy can be added by creating one file and registering it in config
  - Existing strategies are unaffected by adding a new one

4. Reference Strategy: EMA Crossover

- Implement a simple Exponential Moving Average crossover strategy as the first concrete strategy
- Configurable fast/slow EMA periods (defaults: 9 / 21)
- Generate BUY signal when fast EMA crosses above slow EMA
- Generate SELL signal when fast EMA crosses below slow EMA
- **Acceptance criteria:**
  - Given 50 bars of synthetic price data, the strategy produces the correct number of crossover signals
  - Unit test coverage for signal generation logic

5. Risk Manager

- Enforce a maximum number of concurrent open positions (configurable, default: 3)
- Enforce a per-trade maximum notional value (configurable, default: $500 paper money)
- Hard stop: if account equity drops below a configurable threshold (default: 5% drawdown), halt all trading and flatten positions
- **Kill switch:** `Ctrl+C` gracefully closes all open positions and exits

- **Acceptance criteria:**
  - Bot refuses to place a 4th position when 3 are already open
  - Kill switch tested: all positions closed within 10 seconds of signal

6. Structured Logging

- All events (bar received, signal generated, order placed, order filled, error) logged as structured JSON
- Log to both stdout (human-readable format) and a rolling log file
- Trade journal: on session end, write a summary CSV of all trades (entry, exit, P&L, strategy, ticker)
- **Acceptance criteria:**
  - After a session, `trades.csv` exists and contains at least the columns: `timestamp, ticker, side, qty, entry_price, exit_price, pnl, strategy`

7. Backtester

- Run a strategy against historical OHLCV data in a simulated environment
- Report: total return, number of trades, win rate, max drawdown, Sharpe ratio (simplified)
- **Acceptance criteria:**
  - `python bot.py backtest --strategy ema_cross --ticker AAPL --from 2024-01-01 --to 2024-06-01` runs end-to-end and prints a summary

P1 — Nice to Have (fast follows)

- **RSI strategy** — second reference strategy to validate pluggability
- **Webhook alerts** — send a Discord/Slack message on trade execution
- **Position sizing** — volatility-adjusted position sizing (ATR-based) instead of flat notional
- **Multiple tickers simultaneously** — async loop watching a basket of 5-10 tickers at once
- **Equity curve plot** — matplotlib chart saved to file after backtest
- **Basic unit test suite** — pytest coverage for strategy logic, risk manager rules, and order parsing

---

P2 — Future Considerations (design for, don't build yet)

- Live trading toggle (real money) — architecture should make this a config flag, not a refactor
- Strategy optimisation — parameter sweep / walk-forward testing

- Alternative broker (Interactive Brokers, Alpaca international)
- Crypto support (Binance or Coinbase API)
- Web dashboard for live monitoring

Tech Stack Recommendation

Layer	**Recommended**	**Rationale**
Language	Python 3.11+	Best ecosystem for data/finance; you'll be comfortable
Broker API	Alpaca (paper)	Free, clean API, UK-accessible, no PDT rules in paper
HTTP client	`httpx` (async)	Better than `requests` for concurrent feed polling
WebSocket	`websockets` or Alpaca SDK	Real-time price streaming
Data manipulation	`pandas`	OHLCV data, indicator calculation
Indicators	`pandas-ta` or `ta-lib`	Prebuilt EMA, RSI etc. — no need to hand-roll
Scheduling	`asyncio` event loop	Native async — no need for APScheduler for v1
Config	`pydantic-settings` + `.env`	Type-safe config, keeps secrets out of code
Testing	`pytest` + `pytest-asyncio`	Standard; works well with async code
Logging	`structlog`	Structured JSON logging, human-readable in terminal

---

Project Structure

```
trading-bot/
├── bot/
│   ├── __init__.py
│   ├── main.py             # Entry point — live trading loop
│   ├── backtest.py         # Backtesting runner
│   ├── config.py           # Pydantic settings model
│   ├── data/
│   │   ├── feed.py         # Live data feed (WebSocket / REST poll)
│   │   └── historical.py   # Historical OHLCV fetcher
│   ├── strategies/
│   │   ├── base.py         # Strategy protocol / ABC
│   │   ├── ema_cross.py    # EMA crossover strategy
│   │   └── registry.py     # Strategy lookup by name
│   ├── risk/
│   │   └── manager.py      # Risk rules, kill switch
│   ├── execution/
│   │   └── broker.py       # Alpaca order placement
│   └── logging/
│       └── logger.py       # structlog setup, trade journal writer
├── tests/
```

```
│   ├── test_ema_cross.py
│   ├── test_risk_manager.py
│   └── fixtures/
│       └── sample_bars.json
├── .env.example
├── config.yaml             # Tickers, strategy name, params
├── requirements.txt
└── README.md
```

Success Metrics

Metric	**Target**
Bot runs a full session without crashing	100% — non-negotiable
All trade decisions are logged and explainable	100% of trades in journal
Kill switch closes positions	Within 10 seconds
Backtest completes on 6 months of 1-min data	Under 60 seconds
New strategy plugged in without touching core	Yes — validated with RSI strategy (P1)
Test coverage on strategy + risk logic	≥ 80%

Open Questions

Question	Owner	Blocking?
Alpaca paper trading accessible from UK without VPN?	Sam (check Alpaca docs/signup)	Yes — confirm before starting broker layer
Use Alpaca SDK (`alpaca-trade-api`) or raw REST?	Engineering	No — SDK preferred but raw REST is fallback
1-minute bars sufficient or need tick data for accuracy?	Strategy design	No — 1-min fine for EMA/RSI strategies
Where to store logs? Local only, or push to cloud?	Sam preference	No — local is fine for a fun project

---

Phasing

Phase 1 — Core Skeleton (Week 1-2)

Config, broker connection, data feed, logging scaffolding, basic CLI entry point.

Phase 2 — Strategy + Backtest (Week 2-3)

Strategy interface, EMA cross implementation, backtester, trade journal.

Phase 3 — Live Paper Trading (Week 3-4)

Risk manager, kill switch, live loop with real-time data, full session test.

Phase 4 — Polish (Ongoing)

RSI strategy, alerts, equity curve, test suite.

This is a personal learning project. No financial advice is intended or implied. Paper trading only until explicitly decided otherwise.